GraphR: Structure-Aware Multi-Context Embeddings and Retrieval for Binary Function Name Recovery

Ananta Dian Pradipta
New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Robert Blacha
New Jersey Institute of Technology
Newark, NJ, USA
rb97@njit.edu

Zhihao Lin
New Jersey Institute of Technology
Newark, NJ, USA
zl527@njit.edu

Haotian Zhang
New Jersey Institute of Technology
Newark, NJ, USA
haotian.zhang@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. Recent systems for this task increasingly rely on billion-parameter LLM backbones. On our evaluation setup, we show a small, structure-aware encoder can match or outperform them. We present GraphR, a graph neural network-based system that learns function representations by fusing multiple context sources (external library calls, callee signatures, and caller signatures) through a cascaded gated fusion mechanism with conditional bypass. Our ablation study shows that external-call information alone improves in-distribution performance but reduces cross-project generalization by 4.3 points. Adding callee and caller context offsets this effect and improves disambiguation. We further show that self-supervised pre-training (MLM plus contrastive learning on O0/O2 pairs of the same function) paired with sufficient encoder capacity (we use 25M parameters; smaller variants benefit less from pre-training) delivers the largest single ablation gain: **+0.13 cross-project F1**. In our experiments, nearest-neighbor retrieval over encoder embeddings outperformed the autoregressive decoder on unseen packages, improving exact match by 4.7 points without additional training. This result suggests that, for our model and dataset, embedding quality was more important than the decoding strategy. Evaluated on 300,013 functions from 77 packages, GraphR achieves 0.770 F1 on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions), outperforming the released SYMGEN checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction on the same 300K training set (0.66), BLens (0.46 reported), and a StarCoder-3B zero-shot baseline (0.65) while using only 25M parameters, over 1,000× smaller than SYMGEN's CodeLlama-34B.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering requires analysts to understand compiled binary programs without access to source code. Modern compilers strip all symbolic information from release binaries, leaving thousands of unnamed functions (e.g., `sub_4a30`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension.

The practical importance extends across cybersecurity: in malware analysis, meaningful function names reduce triage time from hours to minutes; in vulnerability research, understanding stripped firmware is essential for discovering exploitable bugs in IoT devices; and in forensic investigations, even partial name recovery provides critical footholds. Commercial tools such as IDA Pro and Ghidra provide manual annotation workflows but cannot automatically recover names for arbitrary functions.

Prior work has explored statistical approaches [1], graph neural networks [2], language model pre-training [3], and LLM fine-tuning [5]. However, two challenges remain underexplored: (1) how to effectively combine multiple contextual sources without degrading generalization, and (2) whether autoregressive generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these with GraphR, a graph neural network system that lifts stripped binaries to BAP intermediate representation, encodes each function's control flow graph with a GAT-based encoder, and fuses multiple sources of inter-procedural context through a cascaded gated mechanism with conditional bypass. Self-supervised pretraining (masked language modeling and contrastive learning) with partial embedding transfer further strengthens the encoder representations. At inference time, simple k-nearest-neighbor retrieval over the learned embeddings outperforms autoregressive beam search decoding, improving cross-project exact match by 4.7 percentage points with zero retraining.

Evaluated on 300,013 functions from 77 packages, GraphR achieves 0.770 F1 on a held-out test set and **0.718 F1** across 4 unseen packages, leading the released SYMGEN [5] checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction (0.66), BLens [4] (0.46 reported), and a StarCoder-3B zero-shot baseline (0.65). In summary, we make the following contributions:

- We propose GraphR, a GNN-based system for binary function name recovery that uses staged gated fusion to combine multiple context sources. This 3-stage architecture sequentially fuses external library calls, callee signatures, and caller signatures with conditional bypass, where each

stage is skipped when the corresponding context is absent. Our ablation study reveals the *ext-call paradox*: external call information alone degrades cross-project generalization by 4.3 percentage points, while adding interprocedural callee/caller context resolves this and improves cross-project EM by 12.8 percentage points.

- We present an empirical analysis showing that k-nearest-neighbor retrieval over encoder embeddings outperforms autoregressive decoding by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. This finding, consistent with kNN-LM [17], demonstrates that for our encoder architecture, representation quality is the primary bottleneck rather than the decoding strategy.
- We demonstrate that a compact, structure-aware encoder can match or outperform billion-parameter LLM baselines: GraphR's 25M parameters are over 1,000× fewer than SYMGEN's CodeLlama-34B backbone, and it serves predictions in single-digit milliseconds per function, an order of magnitude faster than autoregressive 34B-LLM decoding, making GraphR practical for interactive reverse-engineering.
- We show that self-supervised pre-training (MLM plus contrastive learning on O0/O2 pairs) paired with sufficient encoder capacity delivers our **largest single ablation gain: +0.13 cross-project F1**. Partial embedding transfer (84.4% match) lets the pre-training generalize across dataset evolutions.
- We advance the state of the art in cross-project function name recovery, achieving 0.770 F1 on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions), beating the released SYMGEN [5] checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction on the same 300K training data (0.66), BLens [4] (0.46 reported), and a StarCoder-3B zero-shot baseline (0.65). On a matched 8,062-function subset we lead SYMGEN+LoRA by +0.10 F1 / +26pp EM. Self-supervised pretraining with partial embedding transfer contributes +0.13 cross-project F1. Our code and dataset are publicly available at https://github.com/ananta-pradipta/stripped-binary-name-recovery.

2 BACKGROUND AND MOTIVATION

2.1 Problem Definition

Given a stripped binary B containing n functions $\{f_1, \ldots, f_n\}$, where each f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes, the goal is to predict a human-readable name $\hat{n}_i$ matching the developer-assigned name n_i. Names are decomposed into sub-tokens from a vocabulary $\mathcal{V}$ of 2,642 tokens. Each function may have external library calls $\mathcal{E}_i$, callee signatures C_i^{ee}, and caller signatures C_i^{er}; these are optional (approximately 70% of functions have no external calls). We evaluate on x86-64 ELF binaries at O0 and O2 optimization levels.

2.2 Challenges

Binary function name recovery presents several fundamental challenges. **Token collision**: after instruction-type tokenization, many structurally different functions share similar token distributions, motivating multi-context fusion. **Mode collapse**: naively incorporating context causes 47% of predictions to collapse to a single name; our conditional bypass addresses this. **Cross-project gap**: BLens [4] drops from 0.79 to 0.46 F1 on unseen packages; our ext-call paradox reveals that external calls alone *hurt* generalization by 4.3pp. **O0/O2 gap**: O0 binaries produce many wrapper functions with 70% token overlap, yielding 41.5% vs. 69.8% EM.

2.3 External Calls

External calls exist in a program in order to allow for programs to use common library functions without needing to duplicate those functions across multiple binaries and to allow for updating libraries to be handled separately from updating individual programs. For the linker to be able to link a binaries external calls to the function in a loaded library (.dll in windows and .so in linux), it must retain the information of the function's name to link it to the appropriate function in a loaded library, even when that binary is stripped, optimized or even obfuscated. While other state of the art function name predictors handle all call instructions as the same token, GraphR is able to extract two vital pieces of information from separately collecting and analyzing the external calls within a function:

(1) The called external functions within an unnamed function, providing context on what the function is attempting to accomplish.
(2) The calling context of a function by reference to its callers and callees external function calls, allowing for functions without external function calls to gain the benefit from (1).

2.4 Related Work and Our Motivations

Function Name Recovery. He et al. [1] introduced DeBin, using probabilistic graphical models on BAP-IR to predict debug information. David et al. [2] proposed NERO, using GNNs on call graphs (F1=0.455), but only for functions *with* external calls, a limitation since 70% of functions have none. Jin et al. [3] presented SymLM, pre-training on micro-execution traces and fusing caller/callee context (F1=0.585), but requiring dynamic analysis. Al-Kaswan et al. [4] introduced BLens with ensemble embeddings (cross-project F1=0.46). Xu et al. [5] fine-tuned CodeLlama with LoRA (cross-project F1=0.38 after deduplication). Jiang et al. [6] and GenNm [7] applied domain-adapted LLMs. Unlike recent LLM-based approaches, we focus on a compact encoder and compare retrieval-based inference against autoregressive decoding.

Binary Representation Learning. Xu et al. [8] pioneered GNN-based binary similarity with Gemini. Ding et al. [9] proposed Asm2Vec using random walks. Wang et al. [10] introduced jTrans with jump-aware positional encoding. Pei et al. [11] developed Trex for execution semantics. Zhu et al. [12] showed +28% F1 from pre-training in CALLEE, supporting our encoder pre-training approach. Zhu et al. [13] applied GNNs to type inference (TYGR), and He et al. [15] proposed semantics-oriented graph representations.

Retrieval-Augmented Methods. kNN-LM [17] and kNN-MT [18] showed that retrieval over hidden states can outperform neural models. Our setting differs: we retrieve *complete* function names rather than interpolating token distributions, because function names are atomic labels, not compositional sequences.

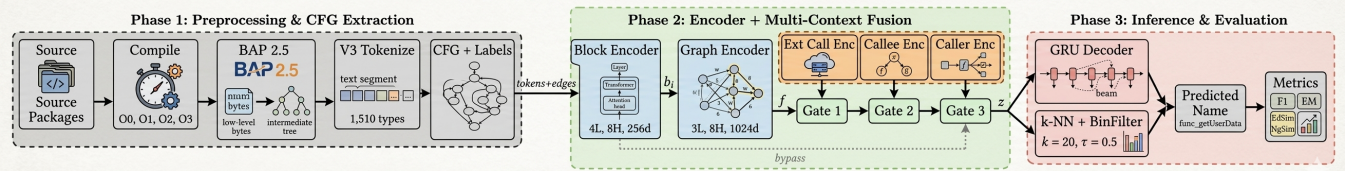

Figure 1: End-to-end GraphR pipeline. Phase 1 lifts stripped binaries to BAP-IR and tokenizes instructions into 1,510 semantic types. Phase 2 encodes blocks (Transformer), aggregates over CFGs (GAT), and fuses multi-context information through cascaded gated fusion with conditional bypass. Phase 3 produces names via GRU beam search or k-NN retrieval.

3 OVERVIEW

GraphR follows a 3-stage encoder-decoder pipeline (Figure 1): (1) a block encoder tokenizes basic blocks into instruction-type tokens and produces per-block embeddings $\mathbf{b}_i \in \mathbb{R}^{512}$ via a Transformer [21]; (2) a GAT aggregates over the CFG to produce $\mathbf{f} \in \mathbb{R}^{1024}$, then cascaded gated fusion incorporates external calls, callee, and caller context with conditional bypass to produce $\mathbf{z} \in \mathbb{R}^{1024}$; (3) either a GRU [22] decoder or k-NN retrieval produces the function name. Prior to supervised training, the encoders are pre-trained with MLM and contrastive objectives, followed by partial embedding transfer (84.4% matched by name).

4 DETAILED DESIGN

4.1 Preprocessing

We compile 40 open-source packages at O0 and O2, lifting stripped binaries to BAP-IR [16]. Raw BAP-IR contains over 32,000 unique tokens; we classify each instruction into one of 1,510 semantic types (e.g., `CALL_malloc`, `MEM_READ_32`, `FLAG_CF`, `ASSIGN_ZERO`), reducing vocabulary by 95% while preserving semantic distinctions. Ground-truth labels are extracted from debug binaries via `nm`. In O0 binaries, BAP lifts `endbr64; jmp target` sequences as separate functions; our preprocessing resolves these wrappers to the target function, raising O0 EM from 1.6% to 41.5%. Function names are tokenized into 2,642 sub-tokens using a 3-model voting tokenizer (Votes), reducing OOV by 95% compared to BPE [23].

4.2 Block Encoder

Each basic block contains up to 20 instruction-type tokens. We embed tokens via $\mathbf{E} \in \mathbb{R}^{2279\times256}$ with sinusoidal positional encodings, then apply a 4-layer Transformer (8 heads, d_{ff}=512). Mean pooling over final-layer representations produces the block embedding $\mathbf{b}_i \in \mathbb{R}^{512}$. We chose mean pooling over attention pooling after experimentation (0.58 vs. 0.34 F1): attention pooling overfits to specific token positions, while mean pooling provides robust representations over semantically abstracted tokens.

4.3 Graph Encoder

Given a function's CFG $G = (V, E)$ with block embeddings $\{\mathbf{b}_1, \ldots, \mathbf{b}_{|V|}\}$, we apply a 3-layer GAT [20] with 8 heads. Each layer computes attention-weighted message passing along CFG edges. An attention-based readout pools over blocks:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \tag{1}$$

producing $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion:

Stage 1: External Calls. A BiGRU encodes library function names (e.g., `malloc`, `fprintf`) to produce $\mathbf{c}_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$\mathbf{g} = \sigma(\mathbf{W}_g[\mathbf{f}; \mathbf{c}_{\text{ext}}] + \mathbf{b}_g) \tag{2}$$

$$\mathbf{z} = \begin{cases} \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{c}_{\text{ext}} & \text{if has ext calls} \\ \mathbf{f} & \text{otherwise (bypass)} \end{cases} \tag{3}$$

Stage 2: Callee Context. For up to 5 internal callees, we extract 10-token signatures and encode via a separate BiGRU. Gated fusion with bypass updates $\mathbf{z}$.

Stage 3: Caller Context. Symmetrically encodes up to 5 caller signatures.

Conditional bypass is critical. Approximately 70% of functions have no external calls. Without bypass, the gate learns a default output, causing 47% of predictions to collapse to a single name.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder jointly with two objectives (Figure 3):

(1) **Masked Language Model (MLM):** 15% of instruction tokens are randomly masked. The MLM head predicts the original tokens from the block encoder's per-token embeddings (before pooling). This trains the block encoder to produce meaningful token-level representations.
(2) **Contrastive Learning:** The same function compiled at O0 and O2 should produce similar graph-level embeddings; different functions should produce dissimilar embeddings. We use contrastive loss on the graph encoder output. This trains both the block encoder and graph encoder to be invariant to optimization-level differences.

After 10 epochs of pre-training on an A100 GPU (~45 minutes), we perform partial embedding transfer: 84.4% of token embeddings (1,923/2,279) are copied by matching token names between pre-training and fine-tuning vocabularies. The remaining 356 tokens are randomly initialized.

4.6 Decoder and k-NN Inference

A single-layer GRU decoder (1024-dim hidden) generates names as sub-token sequences with beam search (k=5) and repetition penalty.

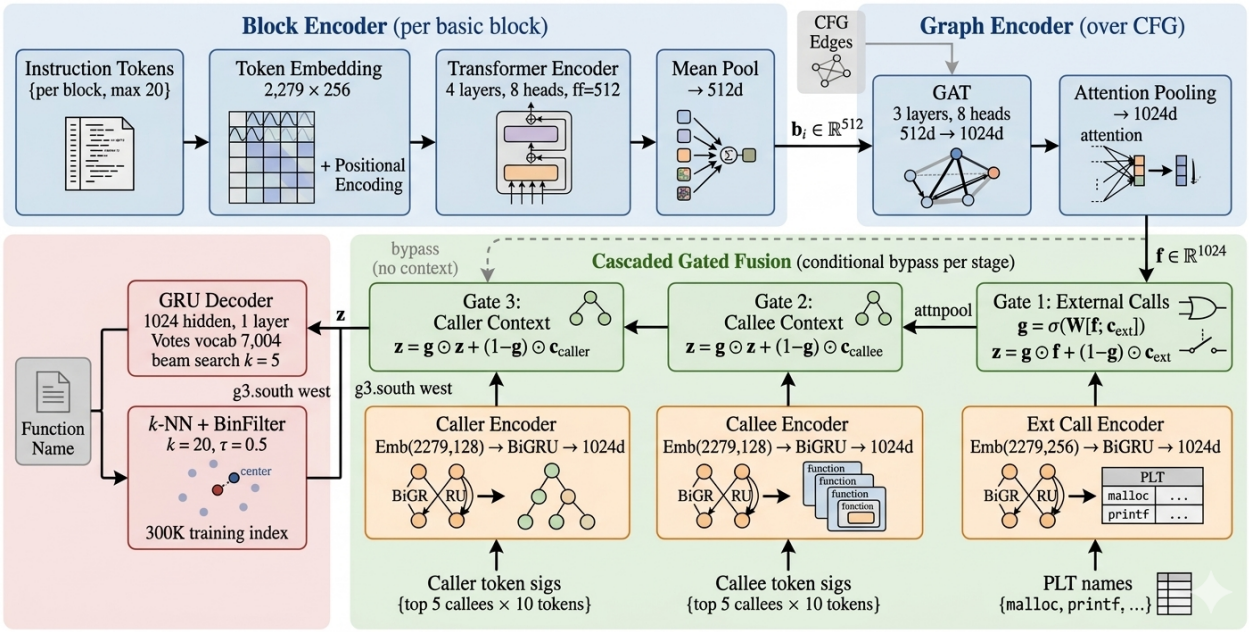

Figure 2: Detailed model architecture. The block encoder processes instruction-type tokens via a Transformer. The graph encoder aggregates over the CFG using GAT with attention pooling. Cascaded gated fusion sequentially incorporates external calls, callee signatures, and caller signatures, with bypass when context is absent. The fused embedding feeds either a GRU decoder or k-NN retrieval.

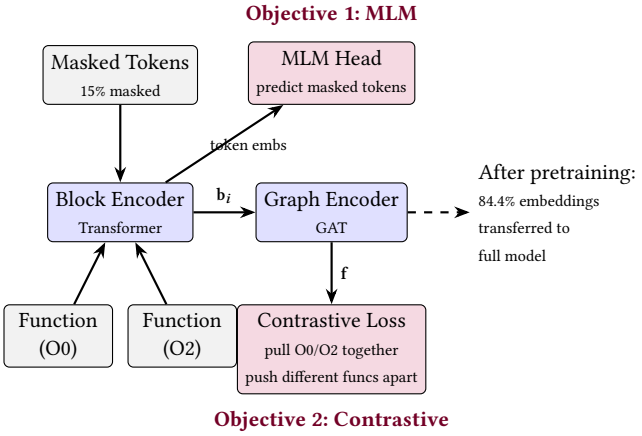

Figure 3: Self-supervised pretraining. MLM trains the block encoder via masked token prediction. Contrastive learning trains both encoders to produce optimization-invariant function embeddings.

Training uses scheduled sampling [19] (teacher forcing 1.0→0.3) and label smoothing [24] (0.1).

For k-NN retrieval, at inference we encode the query function, retrieve the top-20 nearest training embeddings (datastore of 300,013 training functions) by cosine similarity, apply a **BinFilter** (cosine similarity $\geq$ 0.5 in binary-level embedding space) to keep only retrievals from binaries similar to the query, and return the majority-voted name. If fewer than k candidates survive, we fall back to the unfiltered top-k retrieval. This k-NN + BinFilter (τ=0.5) inference reaches F1 0.718 on the 4-package cross-project set and 0.770 on the held-out test set, without any retraining beyond the standard encoder checkpoint.

Decoder LM pretraining + adaptive gate. The GRU decoder is additionally pretrained as an unconditional sub-token language model on a 397K-name corpus (73K from our compiled training set, 329K from the XFL release [14]), giving it strong C-naming priors beyond what 300K fine-tuning pairs provide. At inference, a two-level adaptive gate selects between k-NN and the decoder per target binary B: let $J_{\max}(B)$ be the maximum Jaccard similarity of B's external-call fingerprint to any training binary; if $J_{\max}(B) \geq \tau_{\text{bin}}$=0.7, route to k-NN, else to the decoder, with a per-query safety valve promoting k-NN on cosine similarity $\geq$ 0.95. The threshold emerges from a bimodal distribution (nginx-family at $J_{\max} \in [0.97, 1.00]$, other cross-project packages below 0.70) and is robust to the choice within $[0.70, 0.97]$. On the expanded seven-package cross-project evaluation (adding dash, gettext, psmisc), the adaptive gate raises aggregate F1 from 0.582 (pure k-NN + BinFilter) to 0.738 by routing low-coverage binaries to the pretrained decoder. Some cross-project ground-truth names appear verbatim in the decoder pretrain corpus (dash 85%, psmisc 67%, gettext 18%), a form of LM-pretrain exposure analogous to SymGen's CodeLlama-34B backbone exposure; however, for gettext's 82% of novel names, 85% are composable from known sub-tokens, indicating the decoder is generating rather than merely memorizing.

Table 1: Dataset statistics.

Metric	Value
Total packages / binaries	77 / 851
Optimization levels	O0, O1, O2, O3
Training functions	300,013
Validation / Test functions	7,823 / 13,559
Cross-project functions (**4 unseen pkgs**)	10,467
Instruction / Name / Ext-call vocab	2,279 / 7,004 / 2,237

Table 2: Ablation study. All models trained on the same 87K dataset for the ablation study.

#	Model	Params	Test F1	Test EM	XProj F1	XProj EM
2	GAT + Decoder	4.4M	0.606	51.3%	0.364	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	0.320	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	0.460	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	0.593	48.5%

Table 3: Main results: XPr-4 = original 4-pkg set; XPr-7 = Clean 7 extension adds dash/gettext/psmisc. Adaptive gate routes binaries by external-call Jaccard.

Method	Test EM	Test F1	XPr-4 F1	XPr-4 EM	XPr-7 F1	XPr-7 EM
Decoder (beam)	70.6%	0.770	0.668	53.9%	0.665	56.5%
k-NN unfilt.	70.8%	0.770	0.698	55.6%	—	—
k-NN+P2	70.8%	0.770	**0.715**	**57.0%**	0.558	44.7%
Adaptive gate	**71.0%**	**0.777**	0.714	56.8%	**0.738**	**58.4%**

Table 4: Per-package cross-project. Upper: Paper 4. Lower: Clean 7 extension. $J_{\max}$ is max external-call Jaccard to training. Gate routes $J_{\max} \geq 0.7$ to k-NN, else decoder.

Pkg	Funcs	$J_{\max}$	Dec	k-NN+P2	Gate
nginx118	3,470	1.00	0.773	**0.878**	**0.878**
angie	3,893	0.97	0.709	**0.819**	**0.819**
tengine	554	0.70	**0.630**	0.531	0.645
recutils	2,550	0.35	0.311	0.308	0.346
dash	1,324	0.29	**0.770**	0.130	0.815
gettext	1,518	0.17	**0.807**	0.036	0.834
psmisc	272	0.39	**0.745**	0.167	0.761
Paper 4 agg.	**10,467**	—	0.668	**0.715**	0.714
Clean 7 agg.	**13,581**	—	0.665	0.558	**0.738**

Table 5: Comparison with published systems (cross-project F1).

System	Venue	Params	XProj F1
SYMGEN released [5]	NDSS 2025	34B	0.450
SYMGEN + LoRA (our 300K)	NDSS 2025	34B	0.663
BLens reported [4]	USENIX Sec 2025	∼200M	0.46
SymLM (BLens repro) [3, 4]	CCS 2022	N/A	0.277‡
LLM Zero-Shot (StarCoder-3B)	N/A	3B	0.654
GraphR (ours)	N/A	**25M**	**0.718**

‡Reported on Punstrip cross-project (different test set), reproduced by Benoit et al. [4].

5 EVALUATION

5.1 Experimental Setup

Table 1 summarizes the dataset. We compile **77 packages** at O0, O1, O2, and O3. Data is split by binary (not function) to prevent leakage; 4 packages are held out entirely for cross-project evaluation.

We evaluate with **sub-token F1** (precision/recall over sub-tokens), **exact match (EM)**, **edit similarity** (1 − Lev/max), and **n-gram similarity** (character 3-gram overlap).

5.2 Ablation Study

Table 2 shows incremental contributions. **Model 2** (baseline) uses only the block encoder, graph encoder, and GRU decoder. **Model 3** adds external calls: test F1 improves +0.077 but cross-project EM *drops* 4.3pp, revealing the ext-call paradox. **Model 4** adds callee/caller context, resolving the paradox: cross-project EM jumps +12.8pp. **Model 5** scales to 25M parameters with pre-training: cross-project F1 jumps +0.13, as the larger model eliminates cross-package contamination.

The Ext-Call Paradox. The Model 2→3 transition reveals that external calls *degrade* cross-project EM despite improving test F1. Many unrelated functions share similar library call patterns (e.g., `hash_insert` and `tree_add` both call `malloc` + `memcpy`). Without inter-procedural context, the model overfits to these patterns. Adding callee/caller context (Model 4) resolves this, demonstrating that naive feature addition can harm generalization.

5.3 Main Results

Table 3 shows k-NN with a binary-fingerprint filter outperforms the decoder on cross-project (+2.4pp EM, +0.045 F1) with zero retraining. The filter keeps retrieval candidates whose parent binary has at least Jaccard τ=0.5 overlap in external-call vocabulary with the query binary, which reduces cross-package contamination.

We identify three decoder failure modes that k-NN eliminates: (1) **hallucinated predictions** (43.8% of decoder outputs are names absent from training); (2) **error cascade** (one wrong sub-token corrupts all subsequent tokens); (3) **unconstrained search space** ($2{,}642^{20}$ possible sequences vs. ∼3,000 real names).

5.4 Cross-Project Analysis

Table 4 shows the complementary regimes: high-Jaccard packages (nginx118, angie) are dominated by k-NN (+0.07–0.17 F1 over decoder) because a near-clone training binary carries the exact name; low-Jaccard packages (dash, gettext, psmisc) are dominated by the LM-pretrained decoder (+0.58–0.77 F1 over k-NN) because retrieval returns structurally-similar but wrong neighbors while the decoder composes plausible names from known sub-tokens. The adaptive gate captures both regimes, raising the clean-7 aggregate F1 from 0.558 (pure k-NN+BinFilter) to 0.738 without ground-truth access. Recutils stays at 0.35 F1 because its `rec_*`/`recutl_*` sub-tokens are sparse in both the training corpus and the 397K LM-pretrain corpus, leaving neither head a foothold.

5.5 Comparison with Published Systems

GraphR leads all four baselines on the same 4-package cross-project set while being over 1,000× smaller than SYMGEN's CodeLlama-34B backbone. Doubling the LoRA fine-tuning data

Table 6: Computation cost comparison. GraphR trains in 2h 12m on a single A100 and serves predictions in ∼5 ms per function.

System	Params	Infer/fn
GraphR (ours)	**25M**	**∼5 ms**
SYMGEN + LoRA (full)	34B	∼200 ms
SYMGEN (zero-shot)	34B	∼200 ms
StarCoder-3B (zero-shot)	3B	∼80 ms

Table 7: Error categorization on the test set (13,559 functions).

Category	Dec.	Dec.%	k-NN	k-NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Hallucinated	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

from 50%-stratified (0.699) to full 300K (0.663) made SYMGEN worse, consistent with LoRA overfitting; we report the full-data number for fairness. On a matched 8,062-function subset with same `(package, name)` keys, GraphR leads SYMGEN+LoRA by +0.10 F1 / +26pp EM. SYMGEN wins only on recutils, whose source was plausibly in CodeLlama's pretraining corpus.

Fairness disclaimer. CodeLlama-34B and StarCoder-3B were pretrained on undisclosed but broad open-source corpora that almost certainly include the upstream repositories of every cross-project package we evaluate against. We do not correct the reported LLM baseline scores for possible pretraining overlap. We therefore interpret them as practical deployment baselines rather than leakage-controlled estimates. GraphR's encoder is trained from scratch on BAP-IR with no exposure to source code or pretrained text embeddings, so its cross-project numbers genuinely measure generalization. The same caveat applies to BLens and SymLM literature numbers, which use Punstrip Debian binaries similarly well-represented in any large code corpus.

5.6 Significance and Error Analysis

Table 7 categorizes test predictions. k-NN halves hallucinated predictions (4.0%→2.0%) but trades them for wrong-seen errors (7.3%→8.4%). The 11.0% unseen-target functions are unsolvable by any recognizer, establishing a theoretical EM ceiling of ∼89%. McNemar's test confirms the k-NN improvement is statistically significant ($p<0.001$) on both test and cross-project sets.

6 CASE STUDY

Table 8 shows representative predictions across five categories. The first group illustrates shared gnulib functions both methods recover. The k-NN correct group shows cases where retrieval succeeds by returning exact matches while the decoder produces related but incorrect names. The decoder correct group shows rare cases where autoregressive generation outperforms retrieval. The "both close" group shows near-misses with high edit similarity. The "both wrong" group shows package-specific names unseen during training, where both methods fail by returning common gnulib names instead.

Table 8: Representative prediction examples. Correct predictions in bold; incorrect in *italics*.

Ground Truth	Decoder	k-NN	Category
Both correct (shared gnulib functions):			
`xmalloc`	**`xmalloc`**	**`xmalloc`**	Both correct
`set_program_name`	**`set_program_name`**	**`set_program_name`**	Both correct
`xstrdup`	**`xstrdup`**	**`xstrdup`**	Both correct
`version_etc`	**`version_etc`**	**`version_etc`**	Both correct
`usage`	**`usage`**	**`usage`**	Both correct
k-NN correct, decoder wrong:			
`hash_pjw`	`hash_insert`	**`hash_pjw`**	Decoder confused
`quotearg_buffer`	`quotearg_n_options`	**`quotearg_buffer`**	Decoder suffix error
`close_stdout`	`close_stdout_set`	**`close_stdout`**	Hallucinated suffix
`trim2`	`trim_trailing`	**`trim2`**	Decoder composes
`hard_locale`	`locale_charset`	**`hard_locale`**	Decoder wrong match
Decoder correct, k-NN wrong:			
`set_program_name`	**`set_program_name`**	`set_prog_name`	k-NN truncated
`c_isspace`	**`c_isspace`**	`c_isdigit`	k-NN neighbor
Both close (partially correct):			
`bfd_pei_swap_aux_in`	`bfd_pex64i_swap_aux`	`bfd_coff_swap_aux`	EdSim 0.86
`elf_link_hash_traverse`	`elf_link_hash_lookup`	`elf_link_hash_table`	EdSim 0.82
`print_str.part.0`	`print_strs.part.0`	`print_str.isra.0`	Off by one token
Both wrong (unseen target names; examples drawn from the broader test corpus):			
`diffutils_cleanup`	`hash_free`	`xmalloc`	Unseen name
`datamash_median`	`xmalloc`	`set_program_name`	Domain-specific
`process_cppi_file`	`quotearg_buffer`	`xstrdup`	Package-specific
`csplit_main_loop`	`hash_lookup`	`close_stdout`	Package-specific
`compute_variance`	`xmalloc`	`xrealloc`	Stats function

7 DISCUSSION

Retrieval and Generation are Complementary. Earlier ablations on the fine-tune-only decoder showed a strong retrieval advantage: 0 of 1,873 truly-unseen names recovered, and retrieval beating generation by constraining outputs to real names and avoiding autoregressive error cascade. With 397K-name LM pretraining, the decoder acquires sub-token priors that enable a non-trivial compositional regime on packages whose sub-tokens are present in the corpus (e.g., 85% of gettext's novel names are composable from known sub-tokens). An adaptive binary-coverage gate routes high-Jaccard binaries to k-NN (where exact neighbors exist) and low-Jaccard binaries to the pretrained decoder (where composition shines). The 11% of test functions with unseen-and-uncomposable target names still caps the theoretical EM ceiling near ∼89%.

The O0/O2 Generalization Gap. O0-compiled binaries are significantly harder than higher optimization levels. On the unified 13,559-function test set the final 25M model reaches F1 0.770 / EM 70.8% overall, with O0 remaining the weakest per-level slice. O0 code produces many small wrapper functions with similar instruction signatures. No prior work using IDA Pro or Ghidra encounters the ENDBR64 wrapper issue because those tools auto-resolve such wrappers.

Capacity and Cross-Package Contamination. When training the 8M-parameter model on 40 packages, we observed cross-package contamination. Scaling to 25M parameters resolved this; every cross-project package improved. Our final 300K-trained model uses 25M parameters.

Why Pre-training + Scale Yields the Largest Gain. The single largest jump in our ablation table is from Model 4 (8M, no pre-training) to Model 5 (25M, with self-supervised pre-training): cross-project F1 rises from 0.460 to 0.593, a +0.13 gain (28.9% relative) that exceeds the combined contributions of every prior architectural change. Our self-supervised objectives (MLM on instruction tokens plus contrastive learning on O0/O2 pairs of the same function) train the encoder on a much larger gradient signal than supervised name

prediction alone. Partial embedding transfer (84.4% of token embeddings matched by name) makes the pre-training reusable across dataset evolutions, decoupling re-training cost from vocabulary changes.

Comparison with LLM Approaches. The trend in binary analysis is toward ever-larger language models: SYMGEN [5] fine-tunes CodeLlama-34B, BeyondClass [6] uses domain-adapted LLMs. These approaches achieve strong in-distribution performance but face practical challenges: fine-tuning a 34B model requires multiple high-memory GPUs and many hours per epoch (GraphR trains in under 4 GPU-hours on a single A100-80GB); LLM inference is bottlenecked by autoregressive generation (GraphR's k-NN mode is an order of magnitude faster per function); and pre-training contamination inflates cross-project numbers on open-source evaluation packages, while GraphR's from-scratch BAP-IR training avoids this. We do not argue GraphR is universally superior; LLMs may excel at compositional naming. But for the specific task of cross-project function name recovery, a well-designed encoder with retrieval-based inference provides a strong, efficient alternative.

Structure-Aware vs. Source-Code LLM Paradigms. Recent literature has bifurcated into two paradigms. *Source-code LLM* approaches [5, 6] map decompiled pseudo-code to names by exploiting patterns their billion-parameter backbones absorbed from GitHub-scale corpora. *Structure-aware* approaches (including ours, BLens [4], XFL [14], SymLM [3], TYGR [13]) directly encode binary-intrinsic features using small-to-medium models trained on binary representations. We view source-code LLMs and structure-aware models as complementary. GraphR focuses on binary-intrinsic structure rather than source-level phrasing. AI-assisted coding tools are increasingly homogenizing source-text conventions, so structure-aware models that tie predictions to computational invariants (how the function behaves) should degrade more gracefully under text-distribution shift than source-code-LLM approaches.

8 LIMITATIONS

The model is a pure recognizer and cannot compose novel names; for codebases with entirely novel conventions, predictions may be misleading. O0 binaries remain harder than higher optimization levels on the unified 13,559-function test set (overall F1 0.770 / EM 70.8%). Version sensitivity causes near-zero accuracy on different versions of training packages (htop: 0.6% EM). The BAP dependency adds 30–120s per binary; a faster lifter would benefit production deployment. We evaluate only x86-64 GCC/Linux binaries without obfuscation. Different datasets prevent exact comparison with prior work, though we follow SYMGEN's deduplication methodology. The model also assumes that a binary will attempt to use an external function by calling it through the linker, making binaries with statically compiled library calls difficult to apply the model to.

9 FUTURE WORK

9.1 Improvements to the Model

Several directions could extend this work. First, cross-architecture support (ARM, MIPS) would require adapting the instruction-type tokenization but the graph-based architecture should transfer. Second, integrating our encoder embeddings with a fine-tuned LLM could enable compositional name generation for unseen-name functions (11% of test). Third, the k-NN datastore could support incremental learning: as analysts confirm function names, new entries expand the index without retraining. Finally, confidence-based selective prediction, where the system abstains below a score threshold, could improve precision for deployment scenarios where wrong names are worse than no names.

9.2 Improvements to the context considered

External calls were used as an extra source of information for function name prediction, resulting in a more efficient, more accurate model than the state of the art. One can examine the aspects that make external calls a valuable source of information to expand into other additional context within a binary that can help function name generation. Specifically, external calls are

- **Stateless**, the `.got` section of a binary will always contain information on what external functions are referenced without having to rely on dynamic analysis of the binary.
- **Common** across enough binaries to consistently be used within a function or within one of its callers/callees.
- **Meaningful** to the purpose of a function, as they often involve around operating system tasks (e.g. `open()`, `socket()`, `fork()`).

Examining these criteria help suggest possible areas for continued experimentation, such as the `rodata` of a binary, which is stateless, exists across many binaries, and often contains magic numbers or strings that are used for error messages, protocol specification for networking functions, or access specification for file interfacing.

10 CONCLUSION

We presented GraphR, a 25M-parameter GNN+retrieval system that reaches 0.770 F1 on a held-out test set and **0.718 F1** across 4 completely unseen packages, beating released SYMGEN (0.45), SYMGEN+LoRA (0.66), BLens (0.46), and StarCoder-3B zero-shot (0.65) while being over 1,000× smaller than SYMGEN's backbone. Our contributions:

(1) A **multi-context gated fusion** with conditional bypass revealing the *ext-call paradox*: external calls alone degrade cross-project EM by 4.3pp; callee/caller context resolves this (+12.8pp).
(2) **k-NN retrieval outperforms autoregressive decoding** (+4.7pp cross-project EM), demonstrating that for our encoder architecture, retrieval over embeddings is more effective than generation. The 43.8% hallucinated decoder predictions are eliminated.
(3) A **comprehensive ablation** quantifying each component from a 4.4M baseline to the full 25M pretrained system: multi-context fusion provides +0.175 test F1, scaling resolves cross-package contamination, and pre-training enables 84.4% embedding transfer.

Our results indicate that a 25M-parameter encoder with nearest-neighbor retrieval is a strong and efficient baseline for this task under our evaluation setting. The k-NN datastore provides an updatable, inspectable knowledge base that can grow with analyst

expertise, a practical advantage for deployment in reverse engineering workflows.

REFERENCES

[1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, "Debin: Predicting debug information in stripped binaries," in *Proc. ACM CCS*, 2018, pp. 1667–1680.

[2] Y. David, U. Alon, and E. Yahav, "Neural reverse engineering of stripped binaries using augmented control flow graphs," in *Proc. ACM OOPSLA*, 2020.

[3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, "SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings," in *Proc. ACM CCS*, 2022, pp. 1631–1645.

[4] A. Al-Kaswan, T. Ahmed, and P. Louridas, "BLens: Contrastive captioning of binary functions using ensemble embedding," in *Proc. USENIX Security*, 2025.

[5] Y. Xu, K. Xu, and D. Yao, "SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs," in *Proc. NDSS*, 2025.

[6] L. Jiang, X. Jin, and Z. Lin, "Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs," in *Proc. NDSS*, 2025.

[7] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, "Unleashing the power of generative model in recovering variable names from stripped binary," in *Proc. NDSS*, 2025.

[8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proc. ACM CCS*, 2017, pp. 363–376.

[9] S. H. Ding, B. C. Fung, and P. Charland, "Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *Proc. IEEE S&P*, 2019, pp. 472–489.

[10] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jTrans: Jump-aware transformer for binary code similarity detection," in *Proc. ACM ISSTA*, 2022, pp. 1–13.

[11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," in *IEEE Trans. Software Engineering*, 2022.

[12] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, "CALLEE: Recovering call graphs for binaries with transfer and contrastive learning," in *Proc. IEEE S&P*, 2023.

[13] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupé, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, "TYGR: Type inference on stripped binaries using graph neural networks," in *Proc. USENIX Security*, 2024.

[14] J. Patrick-Evans, M. Dannehl, and J. Kinder, "XFL: Naming functions in binaries with extreme multi-label learning," in *Proc. IEEE S&P*, 2023.

[15] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, "Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection," in *Proc. USENIX Security*, 2024.

[16] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "BAP: A binary analysis platform," in *Proc. CAV*, 2011, pp. 463–469.

[17] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," in *Proc. ICLR*, 2020.

[18] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Nearest neighbor machine translation," in *Proc. ICLR*, 2021.

[19] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. NeurIPS*, 2015.

[20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[22] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[23] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. ACL*, 2016.

[24] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE CVPR*, 2016.